

# PaperPilot — Product Requirements Document

**Author:** Ayush Ojha **Timeline:** 15 days (build alongside LeetCode grind, ~1.5-2 hrs/day)

**Status:** Draft v1

---

## 1. Problem Statement

Reading and comparing multiple research papers is slow. Researchers and students re-read the same sections repeatedly to answer specific questions, and cross-paper comparison (methods, results, claims) requires manually holding multiple documents in memory. There is no lightweight tool that lets someone query a small set of papers directly, get answers grounded in specific sections, and surface contradictions between papers automatically.

## 2. Goals

- Let a user upload 2-5 research papers (PDF) and ask natural-language questions, with answers grounded in specific sections/pages of the source papers.
- Support cross-paper queries: "How does Paper A's method differ from Paper B's?"
- Prove retrieval quality quantitatively, not just qualitatively — this is the single most important differentiator of this project from a generic RAG demo.
- Ship a deployed, working system within 15 days.

## 3. Non-Goals (explicitly out of scope for v1)

- No fine-tuning of any model.
- No multi-agent framework (CrewAI/AutoGen) — a single orchestrator with tool-calling is sufficient.
- No support for arbitrary document types (images, tables-as-images, scanned PDFs) — text-based PDFs only.
- No user accounts / multi-tenancy — single-session use is fine for v1.
- No fancy UI — functional, minimal frontend only.

## 4. Target User

Primary: yourself / fellow students and researchers who want quick, grounded answers across a small paper set — framed in the README as "a research assistant for comparing papers during literature review."

## 5. Core Features (Priority Order — build top-down, cut from the bottom if time runs short)

### P0 — Must ship

1. **PDF ingestion & chunking** — parse PDF, extract text with section/page metadata, chunk with overlap.
2. **Embedding + vector store** — embed chunks (local, no API cost — e.g. `(sentence-transformers)`), store in Chroma or FAISS.
3. **Grounded Q&A** — user asks a question, system retrieves relevant chunks, LLM answers **with citation back to paper + page/section**.
4. **Retrieval evaluation harness** — a fixed set of 15-20 question/answer pairs you write yourself, with known correct source chunks. Script that runs retrieval and reports hit rate (did the correct chunk appear in top-k?). This is your proof of engineering rigor — do not skip or postpone this.

### P1 — Should ship

5. **Tool-calling orchestration** — instead of one fixed retrieval call, the LLM decides which paper(s) to search and can issue multiple retrieval calls per question (e.g., "search paper A," then "search paper B," then synthesize).
6. **Cross-paper comparison** — a mode where the user asks the system to compare methods/claims across two or more loaded papers, using results from multiple tool

calls.

7. **Deployment** — live on Vercel/Render, not just local.

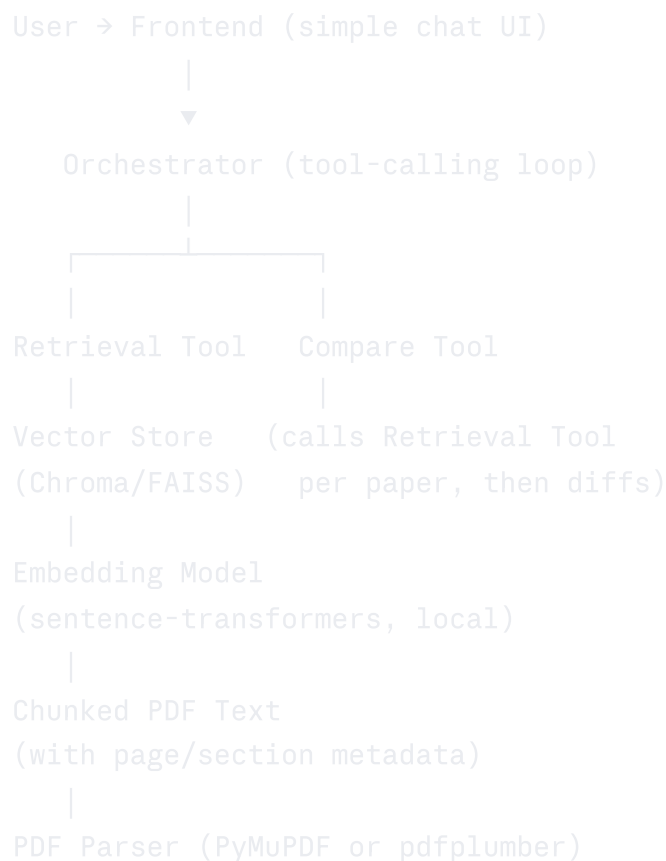
## P2 — Nice to have, cut first under time pressure

8. **Contradiction flagging** — proactively surface when two papers make conflicting claims on a shared topic.

9. **Basic request/latency logging** — log query, retrieved chunks, latency to a file or simple DB table.

10. **Minimal frontend polish** — file upload UI, chat-style Q&A view, citation display.

## 6. Technical Architecture



### Suggested stack:

- **Backend:** Python, FastAPI
- **PDF parsing:** PyMuPDF (`fitz`) — preserves page numbers cleanly
- **Chunking:** recursive character/token splitter with ~200-token overlap; keep page/section metadata attached to every chunk
- **Embeddings:** `sentence-transformers` (e.g. `all-MiniLM-L6-v2`) — free, local, no API cost, fast enough for 2–5 papers
- **Vector store:** Chroma (simplest local setup) or FAISS (if you want to show lower-level familiarity)
- **LLM for answering/orchestration:** Claude or GPT via API, with tool-calling / function-calling
- **Frontend:** simple React or even a Streamlit app if time is tight — functional beats pretty here
- **Deployment:** Render (backend) + Vercel (frontend), or a single Render service if using Streamlit

## 7. Evaluation Plan (the part that makes this "engineering," not "prompt engineering")

1. Pick 2–3 real papers (ideally ones adjacent to your actual research background, so you can speak to them fluently in interviews).
2. Write 15–20 question/answer pairs yourself, each mapped to a specific source chunk/page you know is correct.
3. Build a small eval script: run each question through retrieval, check whether the correct chunk is in top-k (k=3 and k=5), report hit rate.
4. Try at least one variation (e.g., chunk size 256 vs 512, or with/without overlap, or with/without a reranker) and report the difference in hit rate.
5. Put this table directly in your README — this single artifact does more for interview credibility than any additional feature.

Example table to reproduce:

| Config                           | Hit rate @ top-3 | Hit rate @ top-5 |
|----------------------------------|------------------|------------------|
| Chunk size 256, no overlap       | X%               | Y%               |
| Chunk size 512, 50-token overlap | X%               | Y%               |

8. Milestones (15-day plan, ~2 hrs/day)

| Days  | Milestone                                                                         |
|-------|-----------------------------------------------------------------------------------|
| 1-3   | PDF ingestion, chunking, embedding pipeline; basic single-paper retrieval working |
| 4-6   | Multi-paper support + tool-calling orchestrator (LLM picks which paper to query)  |
| 7-9   | Write 15-20 eval Q&A pairs; build eval script; tune chunking based on results     |
| 10-12 | Cross-paper comparison feature                                                    |
| 13-14 | Deploy (Render/Vercel); write README with eval table front and center             |
| 15    | Buffer — something will run over; use this day for it                             |

If behind schedule at day 9: cut comparison (P1) and contradiction-flagging (P2) before cutting the eval harness (P0). The eval is non-negotiable — it's what separates this from a weekend RAG demo.

9. Success Metrics (for the README / resume bullet, not vanity metrics)

- Retrieval hit rate @ top-3 and top-5, with at least one tuning comparison shown.
- Number of papers/tool-calls handled correctly in cross-paper comparison mode (qualitative examples fine if no formal metric).
- Deployed, working link.

10. Risks

- **Time risk:** running in parallel with a 20-day LeetCode grind (July 10-30). Mitigation: strict priority order above; P2 features are explicitly disposable.
- **Scope creep risk:** temptation to add multi-agent frameworks or fancy UI. Mitigation: non-goals section above is final unless there's clear spare time on day 15.
- **Eval subjectivity risk:** self-written Q&A pairs are not a perfectly unbiased eval. Mitigation: acknowledge this explicitly in the README rather than overstating rigor — honesty here reads better to technical interviewers than an inflated claim.

11. Resume Bullet (draft, to refine once built)

Built PaperPilot, a multi-paper research Q&A agent (FastAPI, Chroma, sentence-transformers) with tool-calling orchestration for cross-document comparison; measured retrieval quality via a custom eval harness (15+ Q&A pairs), improving top-3 hit rate from X% to Y% through chunking optimization.